

Chia để trị (Divide and Conquer)

PHỤ THUỘC A/03 cho hệ thức truy hồi; A/04 cho quy nạp — chia để trị và quy nạp là hai mặt của một đồng xu; C/13 cho điểm xuất phát vết cạn.

TÓM TẮT

Chia để trị khai thác một tính chất rất cụ thể: nếu bài toán tách được thành các phần *độc lập* thì tổng chi phí không cộng lại mà sụp xuống theo hệ thức truy hồi. Chương đi qua các đại diện — sắp xếp trộn, quicksort, chọn phần tử thứ k , cặp điểm gần nhất, nhân nhanh — nhưng trọng tâm không phải danh sách thuật toán mà là ba câu hỏi thiết kế: chia ở đâu, ghép thế nào, và chi phí ghép có nhỏ hơn lợi ích chia không. Chương cũng là nơi ta gặp lại ngẫu nhiên hoá lần thứ tư, ở dạng thuyết phục nhất: quicksort với chốt ngẫu nhiên. Nếu chỉ nhớ một điều: phần khó của chia để trị nằm ở bước *ghép*, không ở bước *chia* — và mọi thuật toán hay trong chương này đều là kết quả của việc tìm ra một cách ghép rẻ bất ngờ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **chia để trị** — giải bài lớn bằng cách chia thành các bài con cùng dạng nhưng nhỏ hơn, giải chúng độc lập, rồi ghép kết quả.
- **bước ghép (combine)** — công đoạn tạo lời giải của bài lớn từ lời giải các bài con; đây là nơi chứa gần như toàn bộ độ khó.
- **hệ thức truy hồi** — $T(n) = aT(n/b) + f(n)$; a là số bài con, n/b là kích thước mỗi bài con, $f(n)$ là chi phí chia và ghép. Xem A/03.
- **sắp xếp trộn (merge sort)** — chia đôi, sắp mỗi nửa, rồi trộn hai nửa đã sắp trong $O(n)$. Ổn định, $O(n \log n)$ đảm bảo, nhưng cần bộ nhớ phụ.
- **quicksort** — chọn một chốt, phân hoạch mảng thành nhỏ hơn và lớn hơn, rồi đệ quy hai phía. Không cần bộ nhớ phụ, nhanh trong thực tế, nhưng có trường hợp xấu $O(n^2)$.
- **tính ổn định của sắp xếp** — các phần tử bằng nhau giữ nguyên thứ tự tương đối ban đầu. Quan trọng khi sắp nhiều lần theo nhiều khoá.

CÂU HỎI MỞ ĐẦU

- Q1. Chia đôi rồi giải hai nửa — nghe như vẫn phải làm hết công việc. Vậy tiết kiệm đến từ đâu?
- Q2. Vì sao quicksort thường nhanh hơn sắp xếp trộn trên máy thật, dù nó có trường hợp xấu $O(n^2)$ còn sắp xếp trộn thì không?
- Q3. Bài toán tìm cặp điểm gần nhất trong mặt phẳng: sau khi chia đôi và giải hai nửa, vì sao bước ghép lại *không* tốn $O(n^2)$?
- Q4. Nhân hai số n chữ số theo cách học ở trường tốn $O(n^2)$. Karatsuba xuống $O(n^{1,585})$ chỉ bằng một mẹo đại số. Mẹo đó thay đổi cái gì trong hệ thức truy hồi?
- Q5. Khi nào chia để trị là lựa chọn sai — tức là chia bài ra làm mọi thứ tệ hơn?

Chương trước kết thúc bằng câu hỏi “vết cạn lãng phí ở đâu”. Một câu trả lời rất phổ biến: vết cạn xét mọi cặp phần tử, trong khi phần lớn các cặp đó không liên quan gì tới nhau. Nếu ta chia

dữ liệu thành hai nửa và chứng minh được rằng phần lớn công việc chỉ xảy ra *bên trong* mỗi nửa, thì số cặp phải xét giảm mạnh, và giảm lặp đi lặp lại ở mỗi mức của đệ quy.

Đó là toàn bộ ý tưởng chia để trị, và nó có một đặc điểm dễ bị hiểu sai. Bước “chia” hầu như luôn tầm thường — cắt mảng làm đôi thì có gì khó. Cái khó, và cái tạo ra mọi thuật toán đẹp trong chương này, nằm ở bước *ghép*: làm sao dựng lời giải của bài lớn từ lời giải hai nửa mà không tốn lại $O(n^2)$. Khi bạn đọc mỗi thuật toán dưới đây, hãy đọc nó như câu trả lời cho câu hỏi ghép.

1.1 Ba câu hỏi thiết kế và hệ thức truy hồi

Khuôn chung: chia bài kích thước n thành a bài con kích thước n/b , giải đệ quy, rồi ghép với chi phí $f(n)$. Chi phí tổng là $T(n) = aT(n/b) + f(n)$, và chương A/03 đã cho cách đọc nó bằng hình ảnh “tầng nào nặng”.

Điều đáng rút ra từ hình đó, và nó định hướng toàn bộ việc thiết kế: *nếu chi phí ghép $f(n)$ đủ nhỏ thì tổng chi phí bị chi phối bởi các lá; nếu $f(n)$ lớn thì bị chi phối bởi gốc*. Với sắp xếp trộn, $a = 2$, $b = 2$, $f(n) = n$ — rơi đúng vào trường hợp cân bằng, cho $\Theta(n \log n)$. Thừa số $\log n$ chính là số tầng, và nó là cái giá phải trả cho việc bước ghép tốn tuyến tính.

Từ đó có ba câu hỏi thiết kế cần đặt mỗi khi định dùng chia để trị:

Chia ở đâu? Chia đôi đều là mặc định, nhưng không phải luôn đúng. Quicksort chia theo giá trị chốt nên hai phần có thể lệch; thuật toán chọn phần tử thứ k chỉ đệ quy *một* nhánh; thuật toán cặp điểm gần nhất chia theo toạ độ.

Các bài con có thật sự độc lập không? Nếu lời giải của nửa này ảnh hưởng tới nửa kia thì chia để trị không áp dụng trực tiếp, và bạn đang ở địa hạt quy hoạch động (C/16).

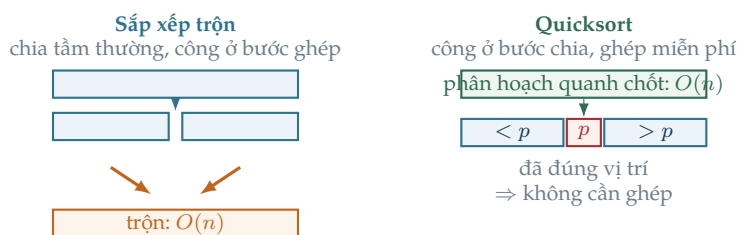
Ghép có rẻ hơn không? Nếu ghép vẫn tốn $O(n^2)$ thì chẳng tiết kiệm gì. Câu hỏi này là chỗ đòi hỏi sáng tạo thật sự, và ba ví dụ dưới đây là ba câu trả lời đẹp.

1.2 Sắp xếp: hai triết lý ngược nhau

Sắp xếp trộn và quicksort đều là chia để trị, nhưng chúng đặt công việc ở hai chỗ ngược nhau — và so sánh đó dạy nhiều hơn bản thân hai thuật toán.

Sắp xếp trộn: chia dễ, ghép khó. Chia là cắt đôi ở giữa, không cần suy nghĩ. Toàn bộ công việc nằm ở bước trộn hai dãy đã sắp, tốn $O(n)$. Kết quả: $O(n \log n)$ *đảm bảo* cho mọi dữ liệu, ổn định, và trộn là thao tác quét tuần tự nên rất hợp cho dữ liệu lớn hơn bộ nhớ (D/24). Cái giá: cần $O(n)$ bộ nhớ phụ.

Quicksort: chia khó, ghép miễn phí. Toàn bộ công việc nằm ở bước phân hoạch — sắp xếp lại mảng sao cho mọi phần tử nhỏ hơn chốt nằm bên trái. Sau khi hai nửa được sắp xong thì không cần ghép gì cả, vì chúng đã ở đúng vị trí. Kết quả: không cần bộ nhớ phụ, truy cập tuần tự nên rất hợp cache, và trong thực tế thường nhanh hơn sắp xếp trộn vài lần. Cái giá: nếu chốt chọn tệ thì hai phần lệch hẳn và chi phí thành $O(n^2)$.



Hình 1.1. Hai cách phân bổ công việc trong chia để trị. Bài học tổng quát: công việc phải nằm ở đâu đó — hoặc lúc chia, hoặc lúc ghép. Khi thiết kế thuật toán chia để trị cho bài mới, hãy thử cả hai hướng: đôi khi chuyển công việc từ ghép sang chia làm mọi thứ đơn giản hẳn.

Chốt ngẫu nhiên: lần thứ tư của cùng một ý tưởng

Trường hợp xấu của quicksort không hiếm: mảng đã sắp sẵn cộng với việc chọn chốt là phần tử đầu tiên cho ngay $O(n^2)$. Và như ở chương B/08 và B/09, cái nguy hiểm là dữ liệu thực tế *rất hay* đã sắp sẵn một phần.

Cách chữa lại là ý tưởng cũ: *chọn chốt ngẫu nhiên*. Khi đó không có dữ liệu vào nào là “xấu”, vì hành vi của thuật toán không còn phụ thuộc thứ tự dữ liệu mà phụ thuộc các lựa chọn ngẫu nhiên của chính nó. Kỳ vọng chi phí là $\Theta(n \log n)$ với *mọi* dữ liệu vào, và xác suất tệ hơn nhiều lần giảm theo hàm mũ.

Đây là lần thứ tư nguyên lý này xuất hiện — bầm phổ dụng, treap, cắt tia qua lấy mẫu, và bây giờ là quicksort. Nếu bạn thấy nó lặp lại thì đó là dấu hiệu nên đọc kỹ chương C/20, nơi nó được phát biểu thành nguyên lý chung.

Chọn phần tử thứ k là biến thể quan trọng của quicksort: sau khi phân hoạch, ta biết chốt đứng ở vị trí nào, nên chỉ cần đệ quy *một* nửa. Chi phí kỳ vọng trở thành $\Theta(n)$ chứ không phải $\Theta(n \log n)$, vì $n + n/2 + n/4 + \dots < 2n$ — lại là chuỗi hình học của chương A/05. Có cả phiên bản tất định đảm bảo $O(n)$ trong trường hợp xấu nhất, dựa trên ý tưởng “trung vị của các trung vị” [1], nhưng hằng số lớn nên trong thực tế bản ngẫu nhiên thường thắng.

1.3 Khi bước ghép đòi hỏi một ý tưởng: cặp điểm gần nhất

Cho n điểm trên mặt phẳng, tìm hai điểm gần nhau nhất. Vết cặn $O(n^2)$.

Chia để trị: sắp các điểm theo hoành độ, chia làm hai nửa bằng một đường thẳng đứng, giải đệ quy mỗi nửa, được khoảng cách nhỏ nhất d trong hai nửa. Bước ghép phải xét các cặp *vắt qua* đường chia — và nếu làm ngây thơ thì đó vẫn là $O(n^2)$.

Ý tưởng cứu bài toán: chỉ những điểm nằm trong dải rộng $2d$ quanh đường chia mới có cơ hội, và trong dải đó, nếu ta sắp theo tung độ thì *mỗi điểm chỉ cần so với một số lượng điểm bị chặn bởi hằng số ở phía sau nó*. Lý do hình học: trong một hình chữ nhật $2d \times d$ không thể có quá một số hằng định điểm mà đôi một cách nhau ít nhất d . Vậy bước ghép là $O(n)$, và tổng chi phí là $O(n \log n)$.

Đây là ví dụ mẫu mực cho luận điểm mở đầu: bước chia tầm thường, còn cái làm nên thuật toán là một quan sát hình học cho phép bước ghép rẻ đi. Khi bạn thiết kế một thuật toán chia để trị mới, đây chính là chỗ phải tìm.

1.4 Nhân nhanh: giảm số bài con thay vì giảm chi phí ghép

Nhân hai số n chữ số theo cách trường học tốn $O(n^2)$. Chia để trị ngây thơ: tách mỗi số thành hai nửa, và tích trở thành bốn phép nhân nửa kích thước cộng vài phép cộng. Truy hồi $T(n) = 4T(n/2) + O(n)$ cho... vẫn $O(n^2)$. Không tiết kiệm gì.

Karatsuba nhận ra rằng *bốn* phép nhân là thừa: ba là đủ, vì tích chéo tính được từ ba tích kia bằng phép cộng trừ. Truy hồi thành $T(n) = 3T(n/2) + O(n)$, cho $O(n^{\log_2 3}) = O(n^{1,585})$.

Ngây thơ a_1b_1 a_1b_2 a_2b_1 a_2b_2 $4T(n/2) \Rightarrow \Theta(n^2)$

Karatsuba a_1b_1 a_2b_2 $(a_1+a_2)(b_1+b_2)$ $3T(n/2) \Rightarrow \Theta(n^{1,585})$

Tích chéo $a_1b_2 + a_2b_1$ suy được từ ba tích kia bằng cộng trừ, nên phép nhân thứ tư là thừa.

Hình 1.2. Hai chiến lược tăng tốc trong chia để trị. Bài cặp điểm gần nhất thắng bằng cách làm *bước ghép* rẻ đi; Karatsuba thắng bằng cách *giảm số bài con*. Vì số bài con nằm ở số mũ của lời giải còn chi phí ghép chỉ cộng thêm, chiến lược thứ hai đổi hăn bậc.

Điểm đáng học ở đây khác hẳn với mục trước, và đó là lý do tôi đặt hai ví dụ cạnh nhau: ở bài cặp điểm gần nhất, ta thắng bằng cách làm *bước ghép* rẻ đi; ở đây, ta thắng bằng cách *giảm số bài con*. Nhìn vào hệ thức truy hồi thì thấy rõ: một bên giảm $f(n)$, một bên giảm a . Và vì a nằm ở số mũ còn $f(n)$ chỉ là cộng thêm, việc giảm a có tác động sâu hơn — nó đổi *bậc* của lời giải.

Cùng ý tưởng cho phép nhân ma trận: Strassen[2] giảm từ 8 xuống 7 phép nhân ma trận con, cho $O(n^{\log_2 7}) \approx O(n^{2,81})$. Và cùng ý tưởng, đẩy tới cùng, cho biến đổi Fourier nhanh với $O(n \log n)$ — công cụ ở chương D/21.

CẠM BÃY

Các thuật toán nhân nhanh có ngưỡng hoà vốn thật sự. Karatsuba chỉ thắng cách trường học khi số đủ lớn — thường vài trăm tới vài nghìn chữ số, tùy cài đặt — vì nó tốn thêm nhiều phép cộng và nhiều lời gọi đệ quy. Strassen còn tệ hơn về mặt này: hằng số lớn, kém ổn định số học, và không thân thiện cache, nên các thư viện đại số tuyến tính chỉ dùng nó ở kích thước rất lớn nếu có dùng. Đây là ví dụ tiêu biểu cho lời cảnh báo ở chương A/03: bậc tiệm cận nhỏ hơn không tự động nghĩa là nhanh hơn.

1.5 Chia để trị ở những nơi không ngờ

Ba biến thể đáng biết vì chúng mở rộng phạm vi áp dụng.

Đếm trong lúc trộn. Bài đếm số nghịch thế giải được bằng cách sửa sắp xếp trộn: khi trộn, mỗi lần lấy một phần tử từ nửa phải trước một phần tử của nửa trái thì ta vừa phát hiện một loạt nghịch thế và đếm được ngay. Mẫu tổng quát: *nếu bài toán đếm các cặp có quan hệ nào đó, hãy thử đếm chúng trong lúc trộn*. Cùng mẫu giải bài đếm cặp có tổng nhỏ hơn ngưỡng, hay đếm cặp đảo ngược có trọng số.

Chia để trị trên trục thời gian. Nếu các đối tượng chỉ “tồn tại” trong một khoảng thời gian, hãy xây một cây phân đoạn trên trục thời gian, đặt mỗi đối tượng vào $O(\log n)$ nút, rồi duyệt sâu cây đó với một cấu trúc có hoàn tác. Đây chính là kỹ thuật đã nhắc ở B/11 cho bài liên thông động ngoại tuyến.

Tối ưu hoá chia để trị cho quy hoạch động. Khi điểm chuyển tối ưu có tính đơn điệu, ta tính các trạng thái theo kiểu chia để trị và giảm được một thừa số n . Chi tiết ở chương C/16; điều đáng nhớ ở đây là chia để trị không chỉ là cách tổ chức thuật toán mà còn là cách tổ chức *việc tính toán* bên trong một thuật toán khác.

1.6 Biên giới hiện nay: $O(n \log n)$ cho nhân số, và số mũ của nhân ma trận

Câu chuyện Karatsuba không dừng ở năm 1960, và phần tiếp theo của nó dạy một điều mà chương A/03 mới chỉ nói bằng lời cảnh báo: một cận có thể vừa đúng vừa vô dụng.

Sau Karatsuba, Toom và Cook tổng quát hoá mẹo ấy thành “chia thành k mảnh, dùng $2k - 1$ phép nhân”, đẩy số mũ xuống gần 1 nhưng không bao giờ tới. Bước nhảy thật sự đến khi người ta bỏ hẳn cách nhìn chia để trị thô và chuyển sang miền tần số: nhân hai đa thức là tích chập, mà tích chập trở thành phép nhân từng điểm sau biến đổi Fourier (D/21). Schönhage và Strassen dùng đúng ý đó năm 1971 để đạt $O(n \log n \log \log n)$, và cùng lúc phỏng đoán rằng đáp án cuối cùng phải là $O(n \log n)$.

Phỏng đoán ấy được chứng minh năm 2021 bởi Harvey và van der Hoeven[3], đăng trên *Annals of Mathematics* — một tạp chí toán học chứ không phải kỷ yếu tin học, điều đã nói lên tính chất của kết quả. Nhưng đây mới là phần đáng học: thuật toán của họ chỉ thắng Schönhage–Strassen từ những kích thước lớn hơn mọi thứ có thể xuất hiện trong máy tính thật. Nó là một thuật toán *thiên hà*: đúng, đẹp, giải quyết một câu hỏi mở nửa thế kỷ, và không bao giờ được cài vào thư viện nào. Giá trị của nó là chốt lại *hình dạng* của câu trả lời, chứ không phải làm chương trình của ai đó nhanh lên. Và chú ý sự bất đối xứng quen thuộc: chưa ai chứng minh được cận dưới $\Omega(n \log n)$ tương ứng, nên khả năng còn nhanh hơn nữa vẫn chưa bị loại trừ (D/24).

Nhân ma trận có cùng cấu trúc câu chuyện nhưng chưa có hồi kết. Strassen[2] mở màn với $O(n^{2.81})$; nửa thế kỷ tiếp theo là một chuỗi cải tiến bằng những kỹ thuật đại số ngày càng phức tạp, tới $\omega < 2.3729$ của Alman và Vassilevska Williams[4], và các bản tiền ấn tới năm 2026 hạ tiếp xuống quanh 2.3712[5] — những con số này chưa qua phản biện nên hãy đọc chúng như tin tức, không như kiến thức đã ổn định. Cận dưới hiển nhiên là $\omega \geq 2$, vì chỉ riêng việc đọc dữ liệu vào đã tốn n^2 ; không ai biết $\omega = 2$ hay không, và đó là một trong những câu hỏi mở lớn nhất của đại số tính toán.

Phần thực hành thì gọn và ngược hẳn với vẻ hào nhoáng ở trên: mọi thuật toán sau Strassen đều thiên hà, và ngay cả Strassen cũng chỉ được dùng ở kích thước lớn vì hằng số, độ ổn định số học và tính thân thiện cache đều kém hơn thuật toán ba vòng lặp đã tối ưu theo khối. Đây là lời nhắc cụ thể nhất mà tôi biết cho luận điểm của chương A/03: bậc tiệm cận là một trong các thước đo, không phải thước đo duy nhất, và giữa “tồn tại thuật toán với số mũ nhỏ hơn” và “chương trình của tôi chạy nhanh hơn” là cả một khoảng cách có thể đo bằng thập kỷ.

1.7 Gốc rễ: một mẹo đại số phá vỡ một phỏng đoán

Sắp xếp trộn xuất hiện rất sớm — nó được ghi nhận trong một bản mô tả của von Neumann từ năm 1945, thuộc những thuật toán đầu tiên được viết cho máy tính có chương trình lưu trữ. Điều đó không ngẫu nhiên: máy tính thời đó dùng băng từ, và trộn hai dãy đã sắp là thao tác gần như duy nhất mà băng từ làm tốt — đọc tuần tự, không nhảy. Nói cách khác, thuật toán sinh ra từ ràng buộc của phần cứng, đúng như câu chuyện B-cây ở chương B/09.

Quicksort có một nguồn gốc rất cụ thể và rất đời. Tony Hoare nghĩ ra nó khoảng năm 1960 khi đang làm việc về dịch máy Nga–Anh: ông cần sắp xếp các từ của một câu để tra chúng trong

từ điển lưu trên băng từ một cách hiệu quả. Ràng buộc bộ nhớ thời đó rất chặt, và đó là lý do quicksort được thiết kế để sắp xếp *tại chỗ* — một quyết định tình cờ hoá ra lại là lý do nó vẫn thắng trên phần cứng hiện đại, nơi việc tránh cấp phát và truy cập tuần tự quan trọng hơn bao giờ hết.

Câu chuyện đáng nhớ nhất trong chương này thuộc về Karatsuba, và nó là một bài học về giá trị của việc nghi ngờ. Năm 1960, Kolmogorov — một trong những nhà toán học lớn nhất thế kỷ — phát biểu tại một buổi seminar phỏng đoán rằng nhân hai số n chữ số bất buộc cần $\Omega(n^2)$ phép toán. Karatsuba, khi đó là một sinh viên, tìm ra thuật toán ba phép nhân chỉ trong khoảng một tuần, bác bỏ phỏng đoán. Điều đáng học không phải mẹo đại số mà là bối cảnh: một phát biểu “chắc chắn phải cần n^2 ” được đưa ra bởi người có uy quyền lớn nhất, và nó sai — vì trực giác về cận dưới rất khó tin cậy khi chưa có chứng minh. Chương D/24 sẽ nói vì sao chứng minh cận dưới lại khó tới vậy.

Cuối cùng, biến đổi Fourier nhanh có một gốc rễ gây bất ngờ: thuật toán mà Cooley và Tukey công bố năm 1965[6] về sau được phát hiện là đã có mặt trong các ghi chép chưa xuất bản của Gauss từ năm 1805, viết bằng tiếng Latin và bị bỏ quên. Bối cảnh thúc đẩy việc tái phát minh năm 1965 rất cụ thể: nhu cầu phát hiện các vụ thử hạt nhân qua tín hiệu địa chấn, một bài toán xử lý tín hiệu quy mô lớn.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1805 — Gauss (không xuất bản)	Nội suy quỹ đạo thiên thể từ các quan sát rời rạc.	Ý tưởng của biến đổi Fourier nhanh, bị bỏ quên trong ghi chép suốt 160 năm.
1945 — von Neumann	Sắp xếp dữ liệu trên băng từ, nơi chỉ đọc tuần tự được.	Sắp xếp trộn — một trong những thuật toán đầu tiên viết cho máy có chương trình lưu trữ; ràng buộc phần cứng sinh ra thuật toán.
1960 — Hoare	Sắp xếp các từ của câu để tra từ điển trên băng, với bộ nhớ rất hạn chế.	Quicksort: sắp tại chỗ. Ràng buộc bộ nhớ thời đó tình cờ tạo ra ưu thế cache của ngày nay.
1960 — Karatsuba	Kolmogorov phỏng đoán nhân số cần $\Omega(n^2)$.	Ba phép nhân thay vì bốn, cho $O(n^{1,585})$; phỏng đoán bị bác bỏ trong một tuần. Bài học: trực giác về cận dưới rất dễ sai.
1965 — Cooley & Tukey	Xử lý tín hiệu địa chấn để phát hiện thử hạt nhân.	FFT $O(n \log n)$; sau đó phát hiện ra Gauss đã có ý tưởng từ 1805.
1969 — Strassen	Nhân ma trận có bất buộc $\Omega(n^3)$ không?	Bảy phép nhân thay vì tám, cho $O(n^{2,81})$; mở ra cả một nhánh nghiên cứu về số mũ nhân ma trận vẫn đang tiếp diễn.

■ 1.8 Liên hệ ngang

■ Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Tính toán song song	MapReduce, phép rút gọn theo cây	Chia để trị là cấu trúc song song lý tưởng vì các bài con độc lập nên chạy đồng thời được. Điều kiện cũng giống hệt: phép ghép phải kết hợp — cùng điều kiện với cây phân đoạn ở B/10.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Xử lý ảnh	Kim tự tháp ảnh, biến đổi wavelet, phương pháp đa lưới	Cùng cấu trúc đệ quy chia đôi độ phân giải. Bài học chung: nhiều bài toán dễ hơn hẳn khi giải ở độ phân giải thô rồi tinh chỉnh dần — một chiến lược mà chia để trị thuần túy không có nhưng rất mạnh trong thực tế.
Đại số tuyến tính số	Phân rã khối, nhân ma trận theo khối	Chia theo khối không phải để giảm bậc mà để <i>hợp cache</i> — mỗi khối vừa trong bộ nhớ nhanh. Đây là chia để trị phục vụ mô hình chi phí bộ nhớ chứ không phục vụ hệ thức truy hồi ($D/24$).
Xử lý tín hiệu	FFT, tích chập nhanh	Ứng dụng nổi tiếng nhất của chia để trị. Ý tưởng chuyển miền — việc khó ở miền này thành việc dễ ở miền kia — cũng chính là mẫu “tổng tiền tố và mảng hiệu” ở B/06, phóng to lên.
Ước lượng trạng thái, SLAM	Chia bản đồ thành các bản đồ con rồi ghép	Cùng khuôn: giải cục bộ rồi ghép. Điểm khác quyết định là bước ghép ở đây <i>không</i> chính xác — các bản đồ con có sai số tương quan — nên việc ghép ngay thơ tạo ra sai lệch hệ thống. Đây là ví dụ cho câu hỏi thứ hai ở mục 1: các bài con có thật sự độc lập không.
Quản lý dự án	Chia nhỏ công việc	Cùng trực giác, nhưng đáng nêu vì lý do thất bại giống nhau: nếu các phần việc phụ thuộc lẫn nhau thì chi phí “ghép” (phối hợp) tăng nhanh hơn lợi ích của việc chia — đúng trường hợp $f(n)$ lớn trong hệ thức truy hồi.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Từ việc phân lớn công việc trong lời giải vét cạn là so sánh những cặp *vắt qua* ranh giới, và chia để trị làm cho số cặp như vậy nhỏ đi rất nhiều. Nói bằng hệ thức truy hồi: nếu chi phí ghép $f(n)$ nhỏ hơn hẳn $n^{\log_b a}$ thì tổng bị chi phối bởi lá, còn nếu bằng nhau thì ta chỉ trả thêm một thừa số $\log n$. So với $O(n^2)$ thì cả hai kết cục đều tốt hơn nhiều bậc. Nếu bước ghép vẫn tốn $O(n^2)$ thì chia để trị không tiết kiệm gì — đó là lý do bước ghép mới là phần khó.
- A2. Vì bậc tiệm cận không nói hết. Quicksort sắp tại chỗ nên không có chi phí cấp phát và không chép dữ liệu qua lại giữa hai mảng; phân hoạch là một lượt quét gần như tuần tự nên rất hợp cache; và vòng lặp trong của nó rất ngắn. Sắp xếp trộn phải ghi ra mảng phụ rồi chép về, tức là chạm mỗi phần tử nhiều lần hơn. Trường hợp xấu $O(n^2)$ thì được xử lý bằng chốt ngẫu nhiên, hoặc bằng cách chuyển sang heapsort khi độ sâu vượt ngưỡng[7].
- A3. Vì một quan sát hình học: chỉ những điểm nằm trong dải rộng $2d$ quanh đường chia mới có thể tạo cặp gần hơn d , và trong dải đó, khi các điểm được sắp theo tung độ thì mỗi điểm chỉ cần so với một số hằng định điểm kế tiếp. Lý do là trong một hình chữ nhật $2d \times d$ không thể nhét quá một số hằng định điểm đôi một cách nhau ít nhất d . Nhờ vậy bước ghép là $O(n)$ thay vì $O(n^2)$.
- A4. Nó giảm số *bài con* a từ 4 xuống 3, chứ không giảm chi phí ghép $f(n)$. Điều này quan trọng vì a nằm ở số mũ của lời giải — $T = \Theta(n^{\log_b a})$ — nên giảm a làm đổi *bậc*, trong khi giảm $f(n)$ chỉ ảnh hưởng khi f đang là thành phần chi phối. Đây là hai chiến lược khác nhau của chia để trị, và bài cặp điểm gần nhất minh họa chiến lược còn lại.
- A5. Ba tình huống. Khi các bài con *không độc lập* — lời giải nửa này ảnh hưởng nửa kia — thì phải chuyển sang quy hoạch động. Khi bước ghép đặt ngang bài gốc, ví dụ vẫn phải xét mọi cặp *vắt ngang*. Và khi n nhỏ, vì chi phí gọi đệ quy và các hằng số phụ vượt quá lợi ích — đó là lý do mọi thư viện sắp xếp thật đều chuyển sang sắp xếp chèn khi đoạn còn dưới vài chục phần tử[8].

NÓI LẠI THẬT ĐƠN GIẢN

Chia để trị là cách bạn xử lý một chồng bài kiểm tra cần chấm: thay vì tự chấm hết, bạn chia đôi chồng bài cho hai người, mỗi người lại chia đôi cho hai người nữa, tới khi mỗi người chỉ còn vài bài. Việc chấm xong rất nhanh. Nhưng có một khâu dễ bị quên: sau khi chấm xong phải *gộp kết quả lại*, và nếu khâu gộp cũng mệt như chấm thì chia ra chẳng lợi gì.

Đó là ý chính của cả chương: chia thì dễ, gộp mới khó, và mọi thuật toán hay ở đây đều là kết quả của việc tìm ra một cách gộp rẻ bất ngờ.

Hai cách sắp xếp kinh điển cho thấy hai kiểu phân bổ công việc. Sắp xếp trộn giống việc bạn cứ chia bừa đôi chồng bài, rồi khi gộp thì phải so từng cặp để trộn hai chồng đã sắp thành một — công việc dồn vào lúc gộp. Quicksort thì ngược: bạn chọn một bài làm mốc và ngay từ đầu chia thành “kém hơn mốc” và “hơn mốc”; sau khi hai bên được sắp xong thì chỉ việc đặt cạnh nhau, không phải gộp gì cả. Công việc dồn vào lúc chia.

Còn mẹo của Karatsuba thì thuộc loại khác hẳn và rất đáng nhớ. Khi nhân hai số lớn, cách chia tự nhiên đòi bốn phép nhân nhỏ. Ông nhận ra chỉ cần ba, vì cái thứ tư suy được từ ba cái kia bằng cộng trừ. Nghe như một mẹo vặt, nhưng vì nó lặp lại ở mọi tầng của đệ quy nên tiết kiệm nhân lên theo cấp số nhân. Đó cũng là bài học chung: trong chia để trị, giảm *số bài con* có tác động sâu hơn nhiều so với làm cho mỗi bài con nhẹ đi một chút.

Một câu: Chia thì dễ; cả nghệ thuật nằm ở chỗ ghép hai nửa lại mà không phải làm lại từ đầu.

Chỗ không rút gọn được: Không có quy trình chung để tìm ra một bước ghép rẻ; nó thường đến từ một quan sát riêng của bài toán, như quan sát hình học trong bài cặp điểm gần nhất.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Ba câu hỏi thiết kế	Chia ở đâu; các bài con có độc lập không; bước ghép có rẻ hơn bài gốc không. Câu thứ ba chứa gần hết độ khó.
●●● Đọc hệ thức truy hồi	$T(n) = aT(n/b) + f(n)$; so $f(n)$ với $n^{\log_b a}$ để biết tầng nào chi phối (Θ).
●●● Sắp xếp trộn so với quicksort	Trộn: chia tầm thường, ghép $O(n)$, đảm bảo $O(n \log n)$, ổn định, cần bộ nhớ phụ. Quicksort: công ở bước phân hoạch, ghép miễn phí, tại chỗ, hợp cache, xấu nhất $O(n^2)$.
●●● Chốt ngẫu nhiên	Làm hành vi phụ thuộc lựa chọn của thuật toán thay vì thứ tự dữ liệu; kỳ vọng $\Theta(n \log n)$ với mọi dữ liệu vào.
●●● Chọn phần tử thứ k	Chỉ đệ quy một nửa sau khi phân hoạch; $n + n/2 + n/4 + \dots < 2n$ cho kỳ vọng $\Theta(n)$.
●●● Hai chiến lược tăng tốc	Giảm chi phí ghép $f(n)$ (cặp điểm gần nhất) hoặc giảm số bài con a (Karatsuba, Strassen). Giảm a đôi <i>bạc</i> vì a nằm ở số mũ.
●● Bước ghép của cặp điểm gần nhất	Chỉ xét dải rộng $2d$ quanh đường chia, sắp theo tung độ, mỗi điểm so với một số hằng định điểm kế tiếp $\Rightarrow O(n)$.
●● Karatsuba	Ba phép nhân thay vì bốn: $T(n) = 3T(n/2) + O(n)$, cho $O(n^{\log_2 3})$. Có ngưỡng hoà vốn thật, chỉ thắng khi số đủ lớn.
●● Đếm trong lúc trộn	Đếm nghịch thế bằng sắp xếp trộn; mẫu tổng quát cho các bài “đếm cặp có quan hệ”.
●● Chia để trị trên trục thời gian	Cây phân đoạn trên trục gian + cấu trúc có hoàn tác; giải bài liên thông động ngoại tuyến ($B/11$).
●● Khi nào chia để trị sai	Bài con phụ thuộc nhau ($\rightarrow$ quy hoạch động); bước ghép đắt ngang bài gốc; hoặc n nhỏ nên chi phí đệ quy vượt lợi ích.
● Bài học Karatsuba–Kolmogorov	Một phỏng đoán cận dưới của nhà toán học lớn bị bác bỏ trong một tuần — trực giác về cận dưới rất dễ sai ($D/24$).
● Biên giới của nhân nhanh	Nhân số nguyên đã có $O(n \log n)$ (2021) nhưng là thuật toán thiên hà; nhân ma trận dừng ở $\omega < 2,3729$ và câu hỏi $\omega = 2$ vẫn mở. Thực hành chỉ dùng tới Karatsuba và Strassen.

■ Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Vì sao các thư viện sắp xếp thật không dùng thuần một thuật toán nào?

| Vì mỗi thuật toán mạnh ở một vùng khác nhau. Cách làm chuẩn — introsort — là bắt đầu bằng quicksort vì nó nhanh nhất trong trường hợp thường; chuyển sang sắp xếp chèn khi đoạn còn nhỏ vì lúc đó hằng số quan trọng hơn bậc; và chuyển sang heapsort khi độ sâu đệ quy vượt ngưỡng, để chặn trường hợp xấu $O(n^2)$ [7]. Ngoài ra còn phải xử lý dữ liệu có nhiều phần tử bằng nhau bằng phân hoạch ba phần[8]. Đây là một trong những ví dụ rõ nhất cho chương E/26.

2. Sắp xếp ổn định có quan trọng không, hay chỉ là chi tiết lý thuyết?

| Rất quan trọng trong thực tế, vì nó cho phép sắp nhiều lần theo nhiều khoá: sắp theo khoá phụ trước, rồi theo khoá chính, và các phần tử bằng khoá chính vẫn giữ đúng thứ tự theo khoá phụ. Đây là cách các bảng dữ liệu và giao diện người dùng làm việc. Sắp xếp trộn ổn định tự nhiên; quicksort thì không, và làm nó ổn định đòi bộ nhớ phụ — một lý do nữa vì sao cả hai cùng tồn tại.

3. Có phải mọi thuật toán chia để trị đều song song hoá được không?

| Về nguyên tắc thì các bài con độc lập nên chạy song song được, nhưng có ba điều kiện thực tế. Bước ghép phải kết hợp và rẻ, nếu không nó thành nút cổ chai. Kích thước bài con phải đủ lớn để bù chi phí tạo luồng — nên trong thực tế người ta chỉ song song hoá vài mức trên cùng rồi chuyển sang tuần tự. Và dữ liệu phải chia được mà không tranh chấp bộ nhớ. Sắp xếp trộn song song hoá tốt hơn quicksort vì bước phân hoạch của quicksort khó song song.

4. Tôi có nên tự cài quicksort trong thi đấu không?

| Không, trừ khi bài yêu cầu một biến thể đặc biệt như chọn phần tử thứ k hoặc phân hoạch ba phần. Thư viện chuẩn đã được kỹ thuật hoá kỹ hơn nhiều so với những gì bạn viết trong năm phút. Điều cần biết là *cạm bẫy*: trong một số ngôn ngữ, hàm sắp xếp chuẩn có trường hợp xấu bị khai thác được, nên khi nghi ngờ hãy trộn ngẫu nhiên mảng trước khi sắp — một dòng code, và nó áp dụng đúng nguyên lý ngẫu nhiên hoá của chương này.

5. Khi nào nên dùng đếm trong lúc trộn thay vì Fenwick cho bài đếm nghịch thế?

| Cả hai đều $O(n \log n)$. Fenwick ngắn hơn nếu bạn đã có sẵn trong thư viện, nhưng cần nén toạ độ. Đếm trong lúc trộn không cần nén và tổng quát hoá tự nhiên sang các quan hệ phức tạp hơn — ví dụ đếm cặp thoả $a_i > 2a_j$, thứ mà Fenwick phải xử lý khéo hơn. Lời khuyên: biết cả hai, và chọn theo việc quan hệ cần đếm có dễ biểu diễn thành “đếm phần tử đã chèn nhỏ hơn x ” hay không.

6. Chia để trị và quy nạp liên quan với nhau thế nào?

| Chúng là hai mặt của một đồng xu, và nhận ra điều đó làm việc viết code lẫn viết chứng minh dễ hơn hẳn. Thuật toán chia để trị giải bài lớn bằng bài nhỏ; chứng minh quy nạp khẳng định tính đúng cho n dựa vào tính đúng cho các giá trị nhỏ hơn. Chúng khớp nhau từng dòng: trường hợp cơ sở của đệ quy ứng với cơ sở quy nạp, và bước ghép ứng với bước quy nạp. Vì vậy nếu bạn viết được thuật toán mà không viết được chứng minh, thường là do bước ghép có lỗ hổng — xem A/04.

7. Ngưỡng chuyển sang thuật toán đơn giản nên đặt bao nhiêu?

| Không có con số phổ quát — nó phụ thuộc ngôn ngữ, kích thước phần tử và máy. Cách đúng là đo: cài cả hai, chạy trên dữ liệu thật với nhiều kích thước, và tìm điểm giao. Trong thực tế các thư viện thường dùng ngưỡng trong khoảng vài chục phần tử, nhưng đừng chép con số mà hãy chép *phương pháp*: đây là một quyết định thực nghiệm, không phải quyết định lý thuyết (E/26).

■ Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Cài sắp xếp trộn và kiểm chứng bằng stress test	Bài tập cài đặt	Viết bất biến của bước trộn ra trước; kiểm với mảng rỗng và mảng toàn phần tử bằng nhau.

Bài	Nguồn	Điều cần nhận ra (tiếp)
• Tìm phần tử thứ k bằng phân hoạch ngẫu nhiên	Bài kinh điển	Chỉ đề quy một nửa; giải thích vì sao kỳ vọng là $\Theta(n)$ chứ không $\Theta(n \log n)$.
•• Đếm số nghịch thế bằng sắp xếp trộn	Bài kinh điển	So với cách Fenwick ở B/10; nêu điều kiện chọn mỗi cách.
•• Lũy thừa nhanh và lũy thừa ma trận	CSES “Exponentiation”[9]	Chia để trị trên số mũ; nối với A/05 và D/21.
•• Tìm cặp điểm gần nhất trong mặt phẳng	CSES “Minimum Euclidean Distance”[9]	Bước ghép là toàn bộ bài; hãy tự chứng minh cận hằng số cho số điểm phải so.
•• Tìm phần tử đa số bằng chia để trị	LeetCode “Majority Element”	So với thuật toán bỏ phiếu $O(n)$ ở A/01 — ví dụ cho thấy chia để trị không phải luôn là cách tốt nhất.
••• Cài Karatsuba cho số lớn và tìm ngưỡng hoà vốn trên máy bạn	Bài tập thực nghiệm	Vẽ đường thời gian theo số chữ số cho cả hai cách; điểm giao là câu trả lời.
••• Bài toán “skyline” — gộp đường viền các toà nhà	LeetCode “The Skyline Problem”	Chia để trị với bước ghép giống trộn; bài mẫu cho việc thiết kế phép ghép.
••• Đếm số cặp (i, j) với $i < j$ và $a_i > 2a_j$	Bài kinh điển	Biến thể của đếm trong lúc trộn; chỗ dễ sai là phải đếm trước khi trộn.
••• Nhân đa thức bằng chia để trị, rồi bằng FFT	Chương D/21	So $O(n^{1,585})$ với $O(n \log n)$ bằng số liệu thật; ngưỡng hoà vốn lại là câu hỏi thực nghiệm.
••• Chia để trị trên trục thời gian cho bài liên thông động ngoại tuyến	Bài thi đấu kinh điển	Kết hợp B/10 và B/11; đây là bài tập tổng hợp tốt nhất của phần B và C.

Kết chương và đọc tiếp

Chia để trị là kỹ thuật đầu tiên trong phần C khai thác cấu trúc *đệ quy* của bài toán. Ba điều đáng mang theo: phần khó nằm ở bước ghép; có hai chiến lược tăng tốc rất khác nhau — làm ghép rẻ đi hay giảm số bài con — và chiến lược thứ hai đối bậc; và điều kiện tiên quyết luôn là các bài con phải *độc lập*.

Chính điều kiện cuối dẫn tới hai chương tiếp theo. Khi các lựa chọn không độc lập mà nối tiếp nhau, có hai kịch bản. Kịch bản may mắn: tồn tại một lựa chọn cục bộ luôn an toàn, và ta có thuật toán tham lam — ngắn tới bất ngờ nhưng cần chứng minh thật sự (C/15). Kịch bản còn lại: không lựa chọn cục bộ nào an toàn, và ta phải xét mọi khả năng nhưng nhớ lại kết quả đã tính — quy hoạch động (C/16).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Harvey & van der Hoeven, “Integer Multiplication in Time $O(n \log n)$ ”	Annals of Mathematics, 2021	Đọc phần mở đầu: nó kể lại toàn bộ lịch sử cận trên của phép nhân và nói rõ vì sao $n \log n$ là ngưỡng được kỳ vọng.
nền tảng	Strassen, “Gaussian Elimination Is Not Optimal”	Numerische Mathematik, 1969	Ba trang, đọc trong hai mươi phút, và là ví dụ hoàn hảo cho chiến lược “giảm số bài con”.
tra khi cần	Alman & Vassilevska Williams, “A Refined Laser Method and Faster Matrix Multiplication”	SODA, 2021	Tra khi muốn biết các cận ω hiện đại được lấy ra bằng kỹ thuật gì; phần mở đầu đủ cho người đọc không chuyên.

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc (tiếp)
tra khi cần	Alman, Duan, Vassilevska Williams, Xu, Xu, Zhou, "Improving the Matrix Multiplication Exponent..."	bản tiền ấn arXiv, 2026 — <i>chưa qua phản biện</i>	Chỉ đọc để biết con số mới nhất; không trích dẫn như kết quả đã ổn định.
tra khi cần	Cooley & Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series"	Mathematics of Computation, 1965	Bài gốc của FFT — công cụ đứng sau mọi thuật toán nhân nhanh hiện đại (D/21).

Nguồn

- [1] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [2] V. Strassen, “Gaussian Elimination Is Not Optimal,” *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.
- [3] D. Harvey and J. van der Hoeven, “Integer Multiplication in Time $O(n \log n)$,” *Annals of Mathematics*, vol. 193, no. 2, pp. 563–617, 2021.
- [4] J. Alman and V. Vassilevska Williams, “A Refined Laser Method and Faster Matrix Multiplication,” in *Proceedings of the 32nd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2021.
- [5] J. Alman, R. Duan, V. Vassilevska Williams, et al. “Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve.” bản tiền ấn arXiv:2608.16884, chưa qua phản biện. (2026), address: <https://arxiv.org/abs/2608.16884> (visited on 09/07/2026).
- [6] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [7] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [8] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [9] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).